

Device Registration Reference

A Step-by-Step Guide for Each Layer of the Stack

Adding a new device to your LoRaWAN network involves coordination across **three independent layers**: the device itself (radio), the gateways (passive forwarders), and the ChirpStack network server (the brain).

This document is the operational reference for that process. It covers what you configure on each layer, what each component does during the OTAA join procedure, what cryptographic keys exist and where they live, and how to troubleshoot when something fails.

Use this as a checklist for every new device or device class you onboard.

Table of Contents

- 1.** Overview — The three-layer responsibility split
- 2.** Key concepts & terminology
 - 2.1 OTAA vs ABP — which to use
 - 2.2 The keys: DevEUI, JoinEUI, AppKey, NwkKey, session keys
 - 2.3 The numbers: DevAddr, FCnt, DevNonce
- 3.** Layer 1 — Device (radio) configuration
 - 3.1 What gets pre-provisioned on the device
 - 3.2 Example: programming a Dragino LHT65N
 - 3.3 Byte-order traps (LSB vs MSB)
- 4.** Layer 2 — Gateway prerequisites
 - 4.1 What the gateway needs to be ready
 - 4.2 Why no per-device gateway config exists
- 5.** Layer 3 — ChirpStack server registration
 - 5.1 The object hierarchy
 - 5.2 Step-by-step: register one device in the web UI
 - 5.3 Bulk registration via the API
- 6.** The OTAA join procedure in detail
 - 6.1 Step-by-step sequence
 - 6.2 Cryptographic details
 - 6.3 RX1 and RX2 windows
- 7.** Verification — confirming the device is working
- 8.** Troubleshooting checklist
- 9.** Master checklist — what to do for every new device

1. Overview — The Three-Layer Responsibility Split

A LoRaWAN device doesn't simply "appear" on the network. Every device must be **pre-provisioned** with identity and security information that *also* exists on the network server. Then a cryptographic handshake (the Join procedure) establishes session keys that protect every subsequent data packet.

The work is distributed across three layers, and one of the most common mistakes is to think it's all done at the server. It isn't:

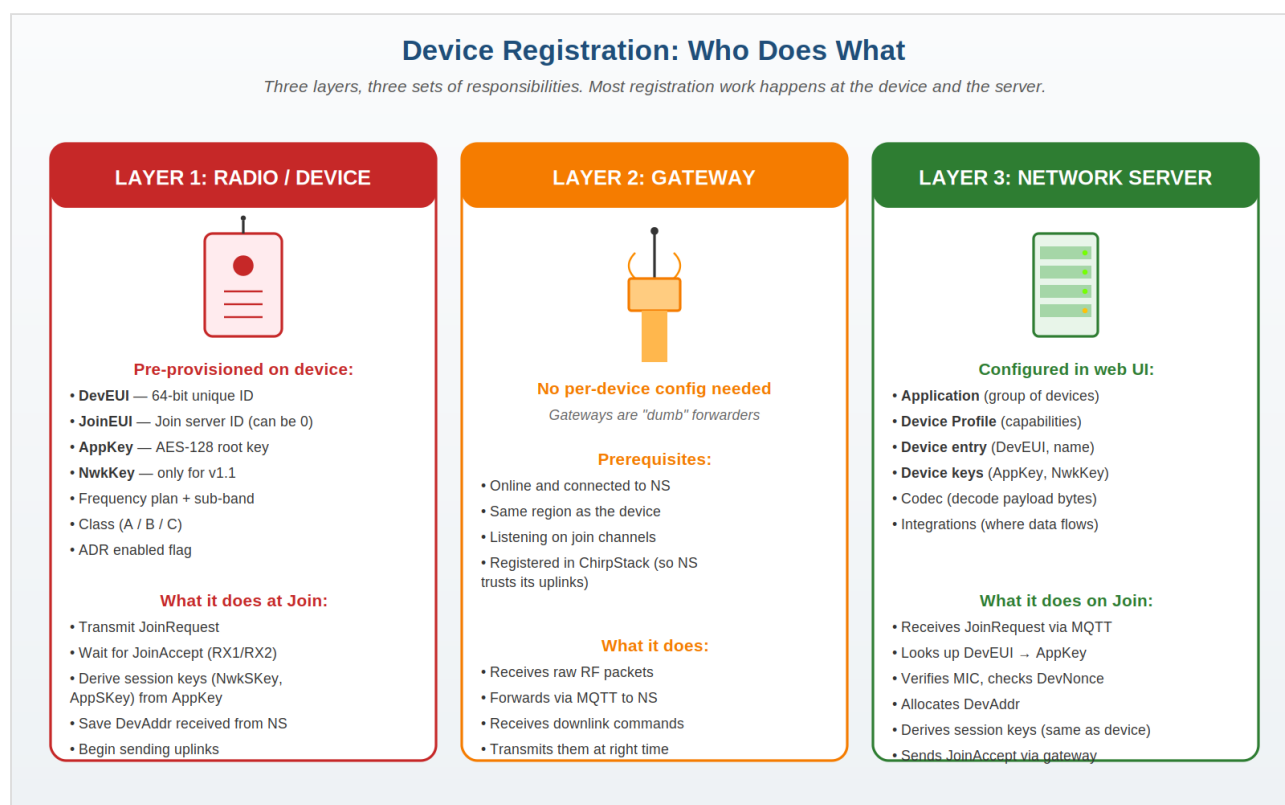


Figure 1 — Most of the device-specific configuration happens at the radio layer (in the device firmware) and at the network server. Gateways are intentionally "dumb" forwarders with no per-device state.

Key principle: The same identity and security material must exist in two places *before* a device can join: programmed into the device firmware, and entered into ChirpStack. If they don't match exactly, the join fails silently.

2. Key Concepts & Terminology

2.1 OTAA vs ABP — which activation mode to use

LoRaWAN supports two ways for a device to "join" the network. The choice affects everything downstream — how you provision keys, how secure the device is, and how you recover from a reboot.

Property	OTAA (Over-The-Air Activation)	ABP (Activation By Personalization)
What's provisioned on device	DevEUI + JoinEUI + AppKey (+ NwkKey for v1.1)	DevAddr + NwkSKey + AppSKey directly
Session keys	Derived per join; rotate when device re-joins	Fixed for device lifetime
Security	Strong — root key never travels	Weaker — session keys are static
Recovery from reboot	Re-joins automatically; new session keys	Must persist frame counter to disk or NS rejects frames
Roaming between networks	Supported	Not supported
Best for	Production deployments (always)	Lab testing, very simple offline devices

Rule: Use OTAA for everything in production. The rest of this document assumes OTAA. ABP is covered briefly in Appendix-style notes where it materially differs.

2.2 The keys

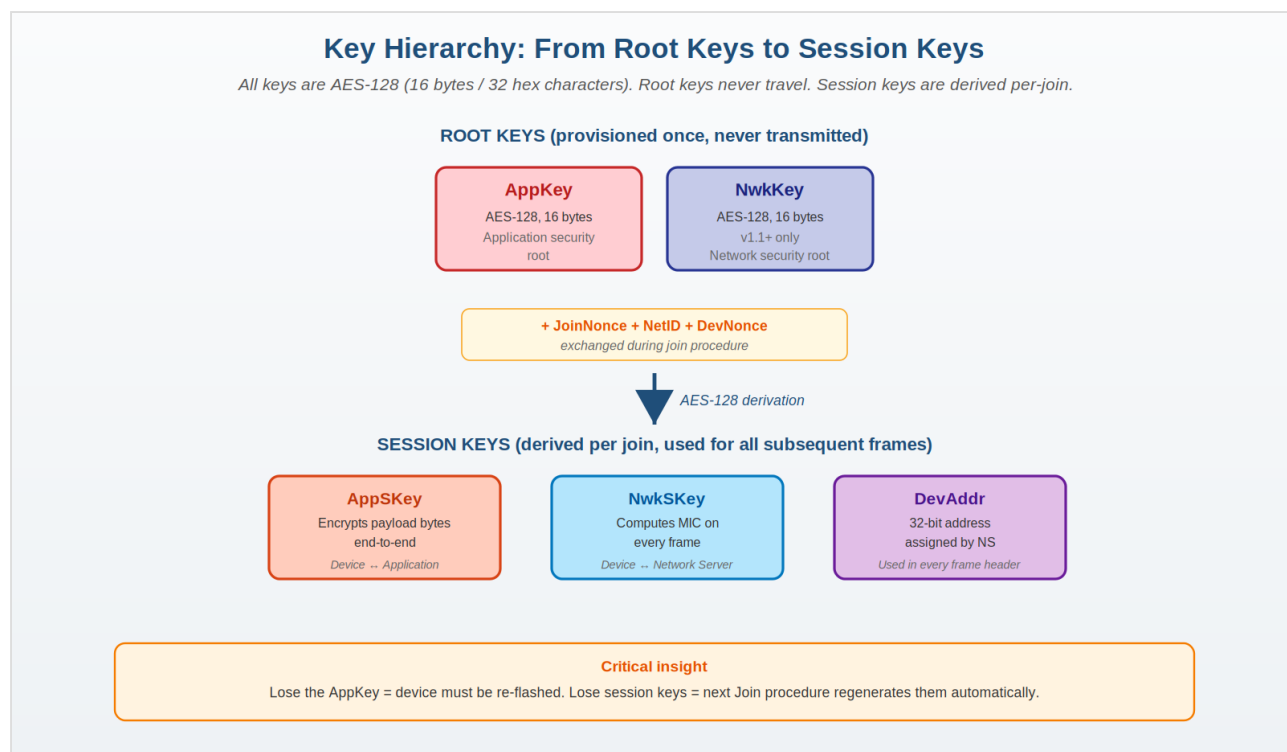


Figure 2 — Root keys (top) are provisioned once and never transmitted. Session keys (bottom) are derived during the join handshake.

Identifier / Key	Size	What it is	Lives where
DevEUI	64 bits	Globally unique device identifier. Like a MAC address.	Device sticker + device firmware + ChirpStack
JoinEUI	64 bits	Identifies the Join Server. For most setups using ChirpStack's internal join server, can be all zeros (0000000000000000).	Device firmware + ChirpStack
AppKey	128 bits	Application root key. Used to derive AppSKey (and authenticate the join in v1.0).	Device firmware + ChirpStack — never on the air
NwkKey	128 bits	Network root key (LoRaWAN v1.1 only). Used to derive NwkSKey variants.	Device firmware + ChirpStack — never on the air
NwkSKey	128 bits	Network session key — signs every uplink with a MIC (message integrity code) for the NS to verify.	Derived per join — in device RAM + NS
AppSKey	128 bits	Application session key — encrypts and decrypts the payload bytes end-to-end.	Derived per join — in device RAM + NS

2.3 The other identifiers

- **DevAddr** — 32-bit network address assigned by the NS during Join. Used in every frame header so the NS can route incoming packets.
- **DevNonce** — A random or counter value sent in every JoinRequest. The NS rejects any JoinRequest with a previously-seen DevNonce (replay protection). A common bug: device firmware that doesn't persist DevNonce across reboots, causing the NS to reject after the first join.
- **FCntUp / FCntDown** — Frame counters incremented on every uplink / downlink. Used as part of the cryptographic input. The NS rejects frames if FCnt goes backwards (replay protection again). This is the #1 cause of "device working then suddenly failing" for ABP devices.
- **JoinNonce** / **AppNonce** — Server-side nonce included in the JoinAccept, used in session-key derivation.
- **NetID** — 24-bit network identifier. For private networks: **000000** is fine.

LAYER 1 3. Device (Radio) Configuration

Before a device can join your network, the following information must be programmed into the device. Where this happens depends on the device type:

- **Commercial sensors (Dragino, Milesight, Adeunis, Decentlab):** Pre-programmed at factory. DevEUI, JoinEUI, AppKey are printed on a sticker or in a packing slip.
- **Development boards (Heltec, TTGO, RAK):** You program the values in via Arduino IDE, MicroPython, or vendor configuration tools.
- **Custom hardware:** You implement provisioning in firmware. Best practice: generate at manufacture, persist in EEPROM or secure element (e.g. ATECC608A).

3.1 What gets pre-provisioned on the device

Item	How to get it	Notes
DevEUI	From sticker, or generated	If burned into hardware (some chips have OTP DevEUI), use that. Otherwise allocate from your own range — private LoRaWAN deployments commonly use locally-administered EUI-64.
JoinEUI	Set to 0000000000000000 for ChirpStack's default internal join server	Only set a different value if you're using an external join server.
AppKey	From sticker for commercial devices; generate randomly for custom devices	Each device should have a unique AppKey. Never re-use across devices.
NwkKey	v1.1 only — same source as AppKey	Skip if your device profile uses LoRaWAN 1.0.x.
LoRaWAN MAC version	From device datasheet	Most current commercial devices: 1.0.3 or 1.0.4. Newer: 1.1.
Regional parameters	Match your gateway region	For Rwanda production: EU868. For US915 bench: sub-band 0 (channels 0-7).
Class (A/B/C)	From device datasheet	Battery sensors: Class A. Always-powered devices needing downlinks: Class C.
ADR enabled	Set ON	Always enable in production. Massively improves capacity.

3.2 Example: programming a Dragino LHT65N

The LHT65N is a popular commercial sensor (temperature + humidity, battery-powered, ~5 years life). It ships with DevEUI/AppKey on a sticker.

To reconfigure, you use AT commands over USB serial:

```
AT+DEUI=a8404117c1862aff      # Set DevEUI
AT+APPEUI=0000000000000000    # Set JoinEUI (zeros for ChirpStack default)
AT+APPKEY=fe2e9c4a7c8541e0a25cf2bee3791e1b  # Set AppKey
AT+NJM=1                      # Set OTAA mode (0=ABP, 1=OTAA)
AT+CLASS=A                    # Set Class A
AT+ADR=1                      # Enable ADR
AT+REJOIN=2                   # Trigger rejoin
AT+CFG                        # Print current config to verify
```

Most commercial devices have a similar AT-command interface, sometimes via Bluetooth instead of USB.

3.3 The byte-order trap

Common bug: Many devices and some legacy tools store EUIs in LSB byte order (little-endian), while ChirpStack always uses MSB (big-endian, the canonical LoRaWAN form). If your DevEUI appears in the sticker as `a8 40 41 17 c1 86 2a ff`, you enter it as that in ChirpStack — but some firmware tools want you to enter it reversed as `ff 2a 86 c1 17 41 40 a8`. Always confirm by checking what arrives in ChirpStack's "LoRaWAN Frames" log when you press the join button. If the DevEUI shown there doesn't match, you have a byte-order mismatch.

3.4 What the device firmware actually does at power-on

1. Reads DevEUI, JoinEUI, AppKey from EEPROM / flash.
2. Picks a join channel (random one from the regional plan).
3. Picks a random DevNonce. Saves it to persistent memory.
4. Constructs JoinRequest, signs MIC with AppKey.
5. Transmits at the join channel and SF (typically SF7 to start, falls back to higher SF if no JoinAccept).
6. Opens RX1 window 5 seconds after transmission — same channel as uplink.
7. Opens RX2 window 6 seconds after transmission — fixed downlink channel (923.3 MHz US915 / 869.525 MHz EU868).
8. If JoinAccept arrives in either window: decrypt with AppKey, derive session keys, save DevAddr, joined.
9. If neither: retry with higher SF or wait, depending on duty cycle constraint.

LAYER 2 4. Gateway Prerequisites

This is the shortest chapter in this document — intentionally. Once a gateway is properly deployed and connected to ChirpStack (covered in *Deployment Record* Phase 2-3 and the *Seeed WM1302 gateway* documentation), **no further per-device configuration happens on the gateway**.

That bears repeating because it's counter-intuitive to engineers from cellular backgrounds: gateways do not know which devices exist. They forward every valid LoRa frame they decode, and ChirpStack figures out which frames belong to which devices.

4.1 What the gateway needs to be ready for a new device to join

Requirement	How to verify
Gateway is online and showing recent stats in ChirpStack	Web UI → Gateways → status = green / Online
Same regional plan as the device	Gateway concentrator region matches device's regional params (e.g. both EU868, or both US915 sub-band 0)
Listening on join channels for the device's region	Default in standard regional plans. For US915 sub-band 0: 902.3 / 902.5 / 902.7 / 902.9 / 903.1 / 903.3 / 903.5 / 903.7 MHz. For EU868: 868.1 / 868.3 / 868.5 MHz.
Physical RF coverage in the device's location	Standing next to the gateway is fine. Outdoors 2-15 km depending on SF and terrain.
Gateway can publish downlinks (not just receive uplinks)	Confirm Mosquitto allows publishes from the gateway EUI on the <code>...command/down</code> topic. Default ChirpStack config does this; only an issue if you've locked down MQTT auth.

4.2 Why gateways are intentionally dumb

This separation of concerns is one of LoRaWAN's strongest design choices, and it has direct operational benefits:

- You can add 10,000 new devices to your network without touching a single gateway.
- Coverage redundancy is free: if 3 gateways hear a device's uplink, all 3 forward it; ChirpStack deduplicates.
- Gateways don't store any device secrets. A compromised gateway leaks no device keys.
- Roaming and ownership are decoupled: a device might be owned by Customer A and Customer A's gateways exist, but if Customer B's gateway hears the device, that frame still routes correctly via the network server.

LAYER 3

5. ChirpStack Server Registration

This is the heart of device onboarding. Everything else either supports or follows from what you configure here.

5.1 The object hierarchy

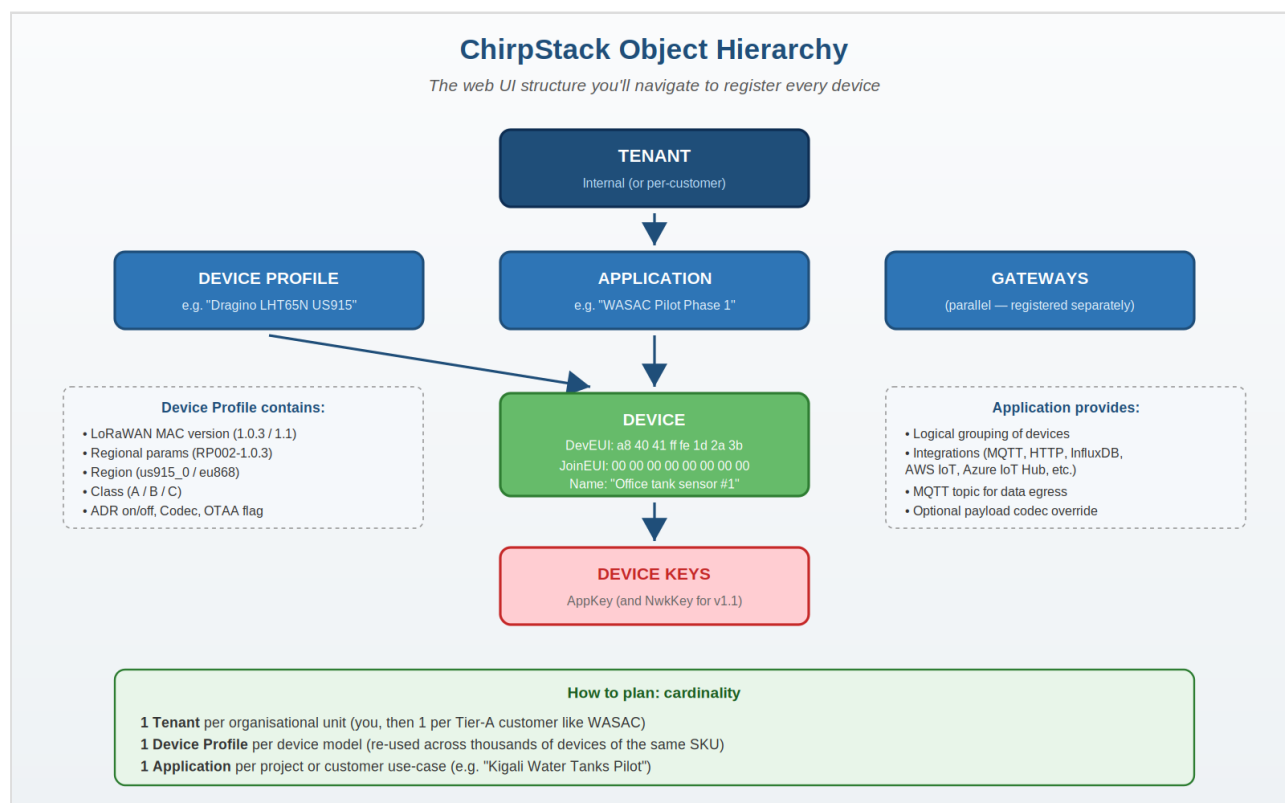


Figure 3 — Tenant → (Device Profile + Application + Gateways) → Device → Device Keys. Get this hierarchy clear in your head before you start clicking.

5.2 Step-by-step: register one device in the web UI

Working assumption: you have ChirpStack running, you've logged in as admin, and you've selected your tenant (e.g. *Internal*).

1 Create a Device Profile (one-time per device model)

Navigate to **Tenant** → **Device Profiles** → **+ Add device profile**.

This step is per device *model*, not per device. You create the profile once for "Dragino LHT65N US915" and reuse it for every LHT65N you onboard later. Fill in:

- **Name:** Something descriptive: `Dragino-LHT65N-EU868`
- **Region:** `EU868` (or `US915 sub-band 0` for bench)
- **MAC version:** Usually `1.0.3` — check device datasheet
- **Regional parameters revision:** `RP002-1.0.3` for matching MAC 1.0.3
- **OTAA:** Enabled (slider on)
- **Class:** A for battery devices
- **ADR:** Enabled
- **Codec (optional but recommended):** Paste the device's payload-decoder JavaScript. Most vendors publish this; for Dragino LHT65N it's available on their wiki.

Click **Submit**.

2 Create an Application (one per use case)

Navigate to **Tenant** → **Applications** → **+ Add application**.

An Application groups devices by use case or customer. For example: `WASAC-Pilot-Phase1` , `HollandGreentech-Greenhouse-Demo` , `Office-Sensors` .

- **Name:** Use a meaningful name — this becomes part of your MQTT topic and shows in dashboards.
- **Description:** Optional context.

Click **Submit**.

3 (Optional but recommended) Configure Integrations on the Application

Click into your new application, navigate to the **Integrations** tab.

Add an integration that defines where your data will flow once devices start uplinking:

- **MQTT** (default, always on) — published to `application/{appID}/device/{devEUI}/event/up`
- **HTTP webhook** — POSTs JSON to your URL on every event
- **InfluxDB** — writes timeseries directly
- **AWS IoT, Azure IoT Hub, GCP Pub/Sub** — cloud-native

For a first device, the default MQTT publishing is usually enough — you'll subscribe with `mosquitto_sub` to verify.

4 Add the Device

From inside the Application, go to the **Devices** tab and click **+ Add device**.

- **Name:** A human-readable identifier, e.g. `tank-sensor-office-1`
- **Description:** Optional — location, what it monitors, install date.
- **DevEUI:** **Exactly** the value programmed into the device. 16 hex characters, no spaces.
- **JoinEUI:** **Exactly** the value programmed into the device. `0000000000000000` in most setups.
- **Device profile:** Pick the one you made in Step 1.
- **Tags:** Optional key-value labels for filtering/grouping (e.g. `location: kacyiru`).
- **Variables:** Optional per-device data the codec or integrations can use.

Click **Submit**. The device appears in the list but is "Never seen" so far.

5 Set the Device Keys

Click the new device, then go to the **Keys (OTAA)** tab.

- **App key:** The 32 hex characters (128 bits) that exactly match what's programmed into the device.
- **Network key (v1.1 only):** Same source — usually the same value as AppKey on simpler devices.

Click **Set device keys**.

If you mistype the AppKey by one character: The device will transmit a JoinRequest, ChirpStack will receive it but fail MIC verification. You'll see the JoinRequest in the LoRaWAN Frames log but no JoinAccept will come back. This is the most common single failure mode.

6 Power on the device and trigger a join

Press the activation button on the device, or power-cycle it. If properly configured, the device will:

1. Send a JoinRequest within ~10 seconds of boot
2. Receive a JoinAccept
3. Send its first uplink (usually a "boot" message containing sensor data)

You should see, in order, the following in the ChirpStack device's **LoRaWAN Frames** tab:

```
JoinRequest      -- from device
JoinAccept       -- from server
UnconfirmedDataUp -- first uplink with sensor data
```

5.3 Bulk registration via the API (for >10 devices)

The web UI works fine for tens of devices. Beyond that, use the REST API or gRPC API. Here's a simple Python example using the REST API:

```

import requests, secrets

CHIRPSTACK_URL = "http://192.168.1.23:8080/api"
API_TOKEN = "your-api-token-from-chirpstack-ui"
APP_ID = "your-application-uuid"
DEVICE_PROFILE_ID = "your-device-profile-uuid"

headers = {
    "Grpc-Metadata-Authorization": f"Bearer {API_TOKEN}",
    "Content-Type": "application/json"
}

# For each device:
for row in csv_rows:
    dev_eui = row["dev_eui"]
    name = row["name"]
    app_key = row.get("app_key") or secrets.token_hex(16)

    # 1. Create device
    r = requests.post(f"{CHIRPSTACK_URL}/devices", json={
        "device": {
            "devEui": dev_eui,
            "joinEui": "0000000000000000",
            "name": name,
            "applicationId": APP_ID,
            "deviceProfileId": DEVICE_PROFILE_ID,
            "isDisabled": False,
            "skipFcmtCheck": False
        }
    }, headers=headers)
    r.raise_for_status()

    # 2. Set device keys
    r = requests.post(f"{CHIRPSTACK_URL}/devices/{dev_eui}/keys", json={
        "deviceKeys": {
            "devEui": dev_eui,
            "nwkJKey": app_key,      # For LoRaWAN 1.0.x, this field carries the AppKey
            "appKey": app_key       # Both fields for compatibility
        }
    }, headers=headers)
    r.raise_for_status()

```

Note: in LoRaWAN 1.0.x the AppKey is stored in the `nwkJKey` field of the ChirpStack v4 API for historical reasons. The `appKey` field is only meaningful for v1.1 devices.

6. The OTAA Join Procedure in Detail

This section explains what actually happens in the ~1 second between the device transmitting its first JoinRequest and it being ready to send data uplinks. Understanding this is essential when troubleshooting joins that don't work.

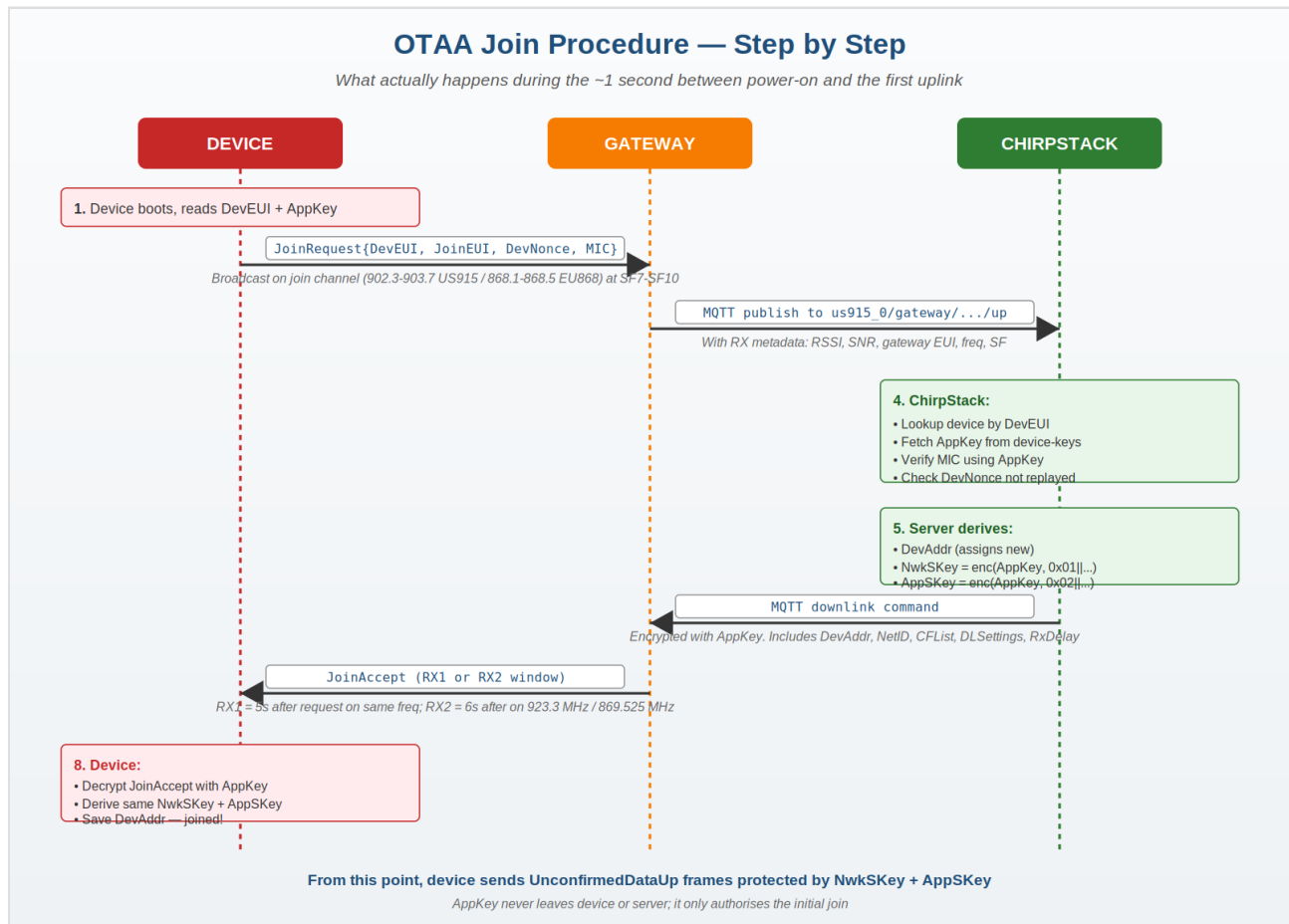


Figure 4 — The complete OTAA join handshake. Note the device, gateway, and server roles at each step.

6.1 Step-by-step sequence

#	Where	What happens
1	Device	Boots. Reads DevEUI, JoinEUI, AppKey from persistent storage. Generates DevNonce.
2	Device	Constructs JoinRequest: <code>MHDR JoinEUI DevEUI DevNonce MIC</code> . MIC is AES-CMAC over the first 4 fields, using AppKey. Transmits on a random join channel at SF7.
3	Gateway	Receives the RF frame on its concentrator. Adds metadata (RSSI, SNR, frequency, time, gateway EUI). Publishes to MQTT topic <code><region>/gateway/<EUI>/event/up</code> .
4	Server	ChirpStack subscribes to the gateway topic. Receives the publish, parses the LoRaWAN frame.
5	Server	Looks up the device by DevEUI in Postgres. Retrieves AppKey from device_keys table.
6	Server	Verifies the MIC by recomputing it with AppKey. If mismatch → silently dropped.
7	Server	Checks DevNonce hasn't been seen before for this device (replay protection). If seen → silently dropped.
8	Server	Allocates DevAddr. Generates JoinNonce. Derives NwkSKey and AppSKey (see 6.2).
9	Server	Constructs JoinAccept: <code>MHDR JoinNonce NetID DevAddr DLSettings RxDelay CFList MIC</code> . Encrypts everything except MHDR with AppKey.
10	Server	Selects the best gateway (highest RSSI) to use for the downlink. Publishes JoinAccept as MQTT command to that gateway, with timing for RX1 or RX2 window.
11	Gateway	Receives MQTT command. Schedules the transmission at the exact specified time (microsecond precision via concentrator).
12	Gateway	Transmits JoinAccept on the air, 5s (RX1) or 6s (RX2) after the original JoinRequest.
13	Device	In its RX window, receives the JoinAccept. Decrypts with AppKey. Verifies MIC.
14	Device	Derives NwkSKey and AppSKey from AppKey + JoinNonce + NetID + DevNonce. Saves DevAddr. Persists session state.
15	Device	Sends first uplink — usually a heartbeat or sensor reading. Encrypted with AppSKey, MIC computed with NwkSKey.

6.2 Cryptographic details (LoRaWAN 1.0.x)

Session keys are derived as follows. Both the device and the network server compute these independently — the AppKey is never transmitted.

```
NwkSKey = aes128_encrypt(AppKey, 0x01 | JoinNonce | NetID | DevNonce | pad16)
AppSKey = aes128_encrypt(AppKey, 0x02 | JoinNonce | NetID | DevNonce | pad16)
```


Where:

- `|` is byte-string concatenation
- `pad16` is zero-padding to make the input 16 bytes total
- `aes128_encrypt` is single-block AES-128 encryption (ECB mode, one block in, one block out)

Note that for LoRaWAN 1.1, the network key (NwkKey) is used to derive three separate keys (FNwkSIntKey, SNwkSIntKey, NwkSEncKey), and the AppKey derives only AppSKey via the Join Server. The structure is more complex but the principle is identical.

6.3 RX1 and RX2 windows — why timing matters

After a device transmits an uplink, it opens two short receive windows. The network server must respond within one of them or the device will not receive the JoinAccept:

Window	Timing after uplink end	Channel	Data rate
RX1	Configurable, default 5 seconds	Same as uplink (frequency-shifted in some regions)	Same SF as uplink (or 1 step lower)
RX2	1 second after RX1 (default 6s after uplink)	Fixed: 869.525 MHz (EU868) / 923.3 MHz (US915)	Fixed: SF12 BW125 (EU868) / SF12 BW500 (US915)

For OTAA joins specifically: Default is `JOIN_ACCEPT_DELAY1 = 5s` for RX1 and `JOIN_ACCEPT_DELAY2 = 6s` for RX2. These are different from normal data-uplink RX1/RX2 timings. If your device's clock drifts more than ~20 ms, the JoinAccept misses the window and the device sees a join failure.

7. Verification — Confirming the Device Works

After power-on, you should be able to verify the device is working end-to-end within 2 minutes. The verification path is:

1. **Did ChirpStack receive the JoinRequest?** → LoRaWAN Frames tab on the device page
2. **Did ChirpStack send a JoinAccept?** → Same tab, next row
3. **Did the device send its first uplink?** → LoRaWAN Frames tab again, third entry
4. **Did the codec parse the payload correctly?** → Device data tab
5. **Did the data reach your application via integration?** → MQTT subscriber output / webhook log

7.1 Quick MQTT subscriber test

On the laptop running ChirpStack:

```
# Install mosquitto-clients if you haven't
sudo apt install mosquitto-clients

# Subscribe to all events for the application
mosquitto_sub -h 192.168.1.23 -t 'application/+/device/+/event/up' -v

# You should see JSON output every time the device uplinks
```

Successful first uplink looks roughly like:

```
application/abc-123/device/a8404117c1862aff/event/up
{
  "deviceInfo": {
    "tenantId": "...",
    "tenantName": "Internal",
    "applicationId": "...",
    "applicationName": "Office-Sensors",
    "deviceProfileName": "Dragino-LHT65N-EU868",
    "deviceName": "tank-sensor-office-1",
    "devEui": "a8404117c1862aff"
  },
  "devAddr": "00ff1234",
  "fCnt": 1,
  "fPort": 2,
  "data": "Ah4AAQAA",
  "object": {
    "temperature": 23.4,
    "humidity": 56.7,
    "battery": 3.2
  }
}
```

The **object** field is populated by the codec. If it's missing, the codec either has an error or wasn't attached to the device profile.

8. Troubleshooting Checklist

When join fails, work through this list in order. Each entry maps to a specific failure mode.

Symptom	Most likely cause	Fix
No JoinRequest in ChirpStack LoRaWAN Frames at all	Device not transmitting / gateway not receiving	Check device LED. Move device next to gateway. Verify gateway is online in ChirpStack.
JoinRequest appears, but DevEUI doesn't match the registered one	Byte-order mismatch between device firmware and ChirpStack	Update one side to match the other. ChirpStack always uses MSB.
JoinRequest received, no JoinAccept sent	MIC verification failed → wrong AppKey on one side	Re-enter AppKey in ChirpStack carefully. Check device's AppKey via AT+APPKEY=? or its config tool.
JoinAccept sent but device doesn't act on it	Device missed RX window (clock drift or DELAY1/2 mismatch)	Confirm device's JOIN_ACCEPT_DELAY1 = 5s default. Check gateway has GPS time sync if available.
"DevNonce already used" error in logs	Device firmware not persisting DevNonce across reboots	Reset the device's stored nonces in ChirpStack (Device → Activation → reset frame counters). Or flash newer firmware that persists DevNonce.
Join works but first uplink rejected with "FCnt validation failed"	Frame counter went backwards (common with ABP after reboot)	For OTAA: re-join, which resets counters. For ABP: enable "skip FCnt check" temporarily (security risk).
Uplink arrives but no decoded fields	Codec missing or has a JavaScript error	Device Profile → Codec tab → check the decoder function compiles and runs without errors. Use the "Test" button.
Device appears online intermittently	Marginal RF link or duty-cycle saturation	Check RSSI in frame metadata. Below -110 dBm is shaky. Add a closer gateway or improve antenna.
Everything looks fine but customer integration isn't getting data	Integration mis-configured at Application level	Application → Integrations tab → check MQTT topic / webhook URL / credentials.

The single most useful debugging tool: The "LoRaWAN Frames" tab in ChirpStack web UI shows everything that passes between server and gateway for this specific device, in real time. Open it in one window, trigger the device action in another, watch the frames appear. 90% of join issues become obvious within 30 seconds of looking at this view.

9. Master Checklist — What to Do for Every New Device

This is the production cheat sheet. Copy it, print it, follow it for every device you onboard.

Before you touch anything

- ☐ Confirm gateway is online and showing recent stats
- ☐ Confirm gateway region matches device region
- ☐ Collect from device sticker: DevEUI, JoinEUI (or default 0), AppKey, LoRaWAN version

On the device

- ☐ Confirm DevEUI is the one on the sticker (or program it)
- ☐ Set JoinEUI to `0000000000000000` (or match what's in ChirpStack)
- ☐ Confirm AppKey is set correctly
- ☐ Set activation mode = OTAA
- ☐ Confirm class (A/B/C) matches profile in ChirpStack
- ☐ Confirm regional plan matches gateway
- ☐ Enable ADR

In ChirpStack web UI

- ☐ Confirm Device Profile exists for this device model (create if first)
- ☐ Confirm Application exists for this use case (create if first)
- ☐ Add device under the Application: DevEUI + JoinEUI + Device Profile + name
- ☐ Set Device Keys: AppKey (and NwkKey for v1.1) — double-check character-by-character
- ☐ Confirm Application Integrations are configured (MQTT, webhook, etc.)

Power on and verify

- ☐ Power on / press join button on device
- ☐ Within 30 seconds, see JoinRequest in LoRaWAN Frames tab
- ☐ Within 6 seconds after that, see JoinAccept
- ☐ Within 1 minute, see first UnconfirmedDataUp
- ☐ Confirm Device data tab shows decoded payload (codec works)
- ☐ Confirm integration receives the data (mosquitto_sub, webhook log)

- ☐ Move device to its final location and confirm RSSI still acceptable (above -115 dBm)

Document the deployment

- ☐ Record DevEUI, location, installation date
- ☐ Note initial battery level (visible in payload for most devices)
- ☐ Note initial SF after ADR has stabilised (usually 24 hours)
- ☐ Add device to your customer-facing dashboard or asset tracker

When this checklist is complete: Your device is fully integrated and producing data. Repeat for the next device. The process scales to thousands of devices — you only change the per-device fields (DevEUI, AppKey, name, location). Everything else is configured once per device model.

*End of reference. Version 1.0 — June 2026.
Companion to the Deployment Record and the Capacity & Scaling Reference.*